Implement Trie (Prefix Tree) (medium)

Problem Statement

Design and implement a Trie (also known as a Prefix Tree). A trie is a tree-like data structure that stores a dynamic set of strings, and is particularly useful for searching for words with a given prefix.

Implement the `Solution` class:

- `Solution()` Initializes the object.
- `void insert(word)` Inserts `word` into the trie, making it available for future searches.
- `bool search(word)` Checks if the word exists in the trie.
- `bool startsWith(word)` Checks if any word in the trie starts with the given prefix.

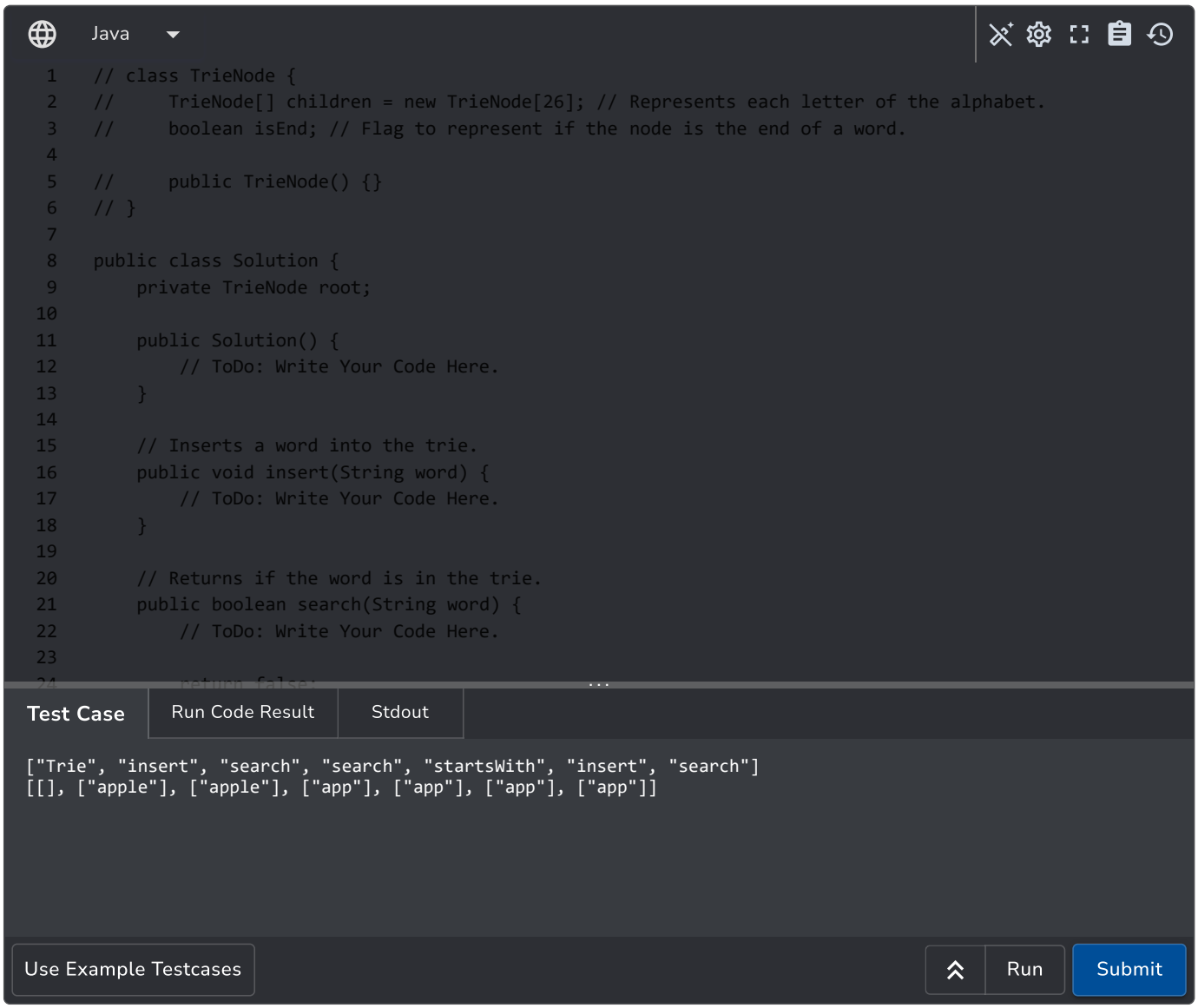
Examples

1. **Example 1:**
   - **Input**:
     - Trie operations: `["Trie", "insert", "search", "startsWith"]`
     - Arguments: `[[], ["apple"], ["apple"], ["app"]]`
   - **Expected Output**: `[-1, -1, 1, 1]`
   - **Justification**: After inserting "apple", "apple" exists in the Trie. There is also a word that starts with "app", which is "apple".
2. **Example 2:**
   - **Input**:
     - Trie operations: `["Trie", "insert", "search", "startsWith", "search"]`
     - Arguments: `[[], ["banana"], ["apple"], ["ban"], ["banana"]]`
   - **Expected Output**: `[-1, -1, 0, 1, 1]`
   - **Justification**: After inserting "banana", "apple" does not exist in the Trie but a word that starts with "ban", which is "banana", does exist.
3. **Example 3:**

- **Input**:
  - Trie operations: `["Trie", "insert", "search", "search", "startsWith"]`
  - Arguments: `[[], ["grape"], ["grape"], ["grap"], ["gr"]]`
- **Expected Output**: `[-1, -1, 1, 1, 1]`
- **Justification**: After inserting "grape", "grape" exists in the Trie. There are words that start with "grap" and "gr", which is "grape".

Try it yourself

Try solving this question here:



← Previous

Introduction To Trie

Next →

Solution: Implement Trie (Prefix Tree)

Mark as Completed